

Tasks Module — Documentation

Field-task (geo / on-site visit) management for EHRMS.

EHRMS · app_backend + hrms (Flutter) · Generated 2026-06-26

A task is a job assigned to a field staff member to visit a **customer** location, travel there with live GPS tracking, prove arrival (photo / selfie / OTP / form), and complete the visit. This document covers both halves of the feature:

- **Backend** — ehrms/app_backend (Node / Express / Mongoose)
- **Mobile app** — ehrms/hrms (Flutter)

1. Concepts & data model

A task's data is deliberately split across **three** Mongo collections so the hot `tasks` collection stays small while the heavy detail/history lives elsewhere.

Collection	Model	Holds
<code>tasks</code>	Task.js	Minimal fields only: title, status, who/whom, dates, exit history
<code>task_details</code>	TaskDetails.js	Full detail: locations, photo proof, OTP, selfies, trip metrics, progress steps
<code>task-settings</code>	TaskSettings.js	Per-company workflow flags (auto-approve, OTP, approval-on-complete)

Live GPS points are **not** stored on the task — they go to the separate `trackings` collection (see `Tracking` model) and are joined back in at read time.

1.1 The two IDs (important)

There are two identifiers and they are easy to confuse:

Field	Format	Meaning
<code>tasks._id</code>	Mongo <code>ObjectId</code>	The real primary key. Used in all API paths <code>/tasks/:id</code> .
<code>tasks.taskId</code>	<code>TASK-XXXXXXXX-XXXX</code>	Human-readable label shown in the UI. Auto-generated on create.
<code>task_details.taskId</code>	<code>ObjectId</code>	= <code>tasks._id</code> — the lookup key linking detail → task. NOT the human label.
<code>trackings.taskId</code>	<code>ObjectId</code> (tasks._id)	Links GPS points back to the task.

`taskId` is **never** accepted from the client on create/update — the server always generates `TASK-XXXXXXXX-XXXX` (see `generateTaskId()` and the `pre('save')` hook in `Task.js`).

1.2 Why the split + the merge

- **Write path:** the controller keeps `tasks` minimal. `updateTask` always `$unset`s the extended keys from `tasks` (`buildUnsetExtended()`) and upserts the full document into `task_details` (`upsertTaskDetails()`).
- **Read path:** `mergeTaskWithDetails()` re-joins `tasks` + `task_details` , then `mergeTaskSettings()` overlays the company's `task-settings` flags into `customFields` so the app sees one flat object.

A one-time backfill script exists for legacy data: `migrate-tasks-to-minimal.js` — moves extended fields out of `tasks` into `task_details` .

1.3 Status lifecycle

Statuses are stored with loose casing/spelling for backward compatibility and **normalized at read time** (`normalizeStatusForApp`) so the app never sees "Unknown". Canonical app-facing statuses:

```
assigned → approved/staffapproved → in_progress → arrived → completed
                                     |           |
                                     exited    waiting_for_approval
                                     hold       (if requireApprovalOnComplete)
                                     exitedOnArrival / holdOnArrival / reopenedOnArrival
reopened ← (admin) ← completed / rejected
```

- **exit vs hold:** `task_exit.status = 'hold'` → staff can resume themselves; `'exited'` → can only resume after an admin reopens.
- Transitions are validated server-side by `VALID_TRANSITIONS` / `isValidStatusTransition` in `taskController.js`. Unknown targets are allowed through (backward compat).
- The app maps raw spellings → status groups for filtering; see `STATUS_GROUP_SPELLINGS` (backend) and `Task.statusFromJson` (app) — these two must stay in sync, since the group decides the card badge.

2. Backend

2.1 Files

File	Responsibility
<code>src/models/Task.js</code>	Minimal task schema + taskId generation
<code>src/models/TaskDetails.js</code>	Full detail schema (<code>strict: false</code>)
<code>src/models/TaskSettings.js</code>	<code>task-settings</code> collection
<code>src/controllers/taskController.js</code>	All task logic + merge/split helpers
<code>src/routes/taskRoutes.js</code>	Route table + multer upload config
<code>src/utils/trackingTaskCount.js</code>	Distinct-tasks-per-day count (timezone-aware)
<code>scripts/migrate-tasks-to-minimal.js</code>	One-time data migration

Some shared helpers are **exported for the tracking controller** to reuse: `buildUnsetExtended` , `upsertTaskDetails` , `computePersistedTravelMetrics` , `normalizeTravelActivityDuration` . The arrive/exit/restart flows live in the tracking controller and write the same `task_details` .

2.2 REST API

Base path: `/tasks` . `protect` = JWT auth middleware.

Method	Path	Auth	Purpose
GET	<code>/tasks</code>	—	All tasks (merged)
GET	<code>/tasks/staff/:staffId</code>	—	Tasks for one staff
GET	<code>/tasks/staff/:staffId/paginated</code>	—	Paginated + search + date range + status-group filter
GET	<code>/tasks/:id</code>	—	One task (merged, with battery + travel metrics)
GET	<code>/tasks/:id/completion-report</code>	—	Task + timeline + route points + form responses
GET	<code>/tasks/:id/tracking-path</code>	✓	Full GPS path for admin replay
POST	<code>/tasks</code>	✓	Create task (auto taskId, businessId from staff)
PATCH	<code>/tasks/:id</code>	✓	Update status/locations/trip; validates transition
POST	<code>/tasks/:id/location</code>	✓	Broadcast live GPS over Socket.io (not persisted here)
PATCH	<code>/tasks/:id/steps</code>	✓	Toggle progress steps
POST	<code>/tasks/:id/photo</code>	✓	Upload photo proof (→ DigitalOcean)
POST	<code>/tasks/:id/selfie</code>	✓	Upload check-in/check-out selfie (→ DigitalOcean)
POST	<code>/tasks/:id/send-otp</code>	✓	Email a 4-digit OTP to the customer
POST	<code>/tasks/:id/verify-otp</code>	✓	Verify OTP, set <code>progressSteps.otpVerified</code>
POST	<code>/tasks/:id/end</code>	✓	Complete the task (or <code>waiting_for_approval</code>)

Arrive / exit / restart are handled by the **tracking** routes (`/tracking/arrived` , `/tracking/exit` , `/tracking/restart`), not these.

2.3 Key behaviours

- **Create** (`createTask`): strips client `taskId` , generates `TASK-...` , resolves `businessId` (staff → auth `companyId` → body), sets `assignedBy` / `assignedDate` , saves minimal task, then upserts full `task_details` .
- **Progress steps**: `reachedLocation` , `photoProof` , `formFilled` , `otpVerified` (+ `checkinCustomerPlace` / `checkoutCustomerPlace` written by selfie upload).
- **Settings overlay** (`mergeTaskSettings`): `enableOtpVerification` → `customFields.otpRequired` ; `requireApprovalOnComplete` and `autoApprove` surfaced to the app. OTP-required comes **only** from settings, never from the task.

- **Travel metrics:** trip duration and drive/walk/stop split are **recomputed at read time** from `trackings` (`computeTravelMetricsFromTrackingRecords`) using the window from start/restart → arrival, so they stay correct even if the client value drifts. A movement-debounce (≥ 1 min sustained) avoids noisy timeline entries.
- **Battery:** key events (start, arrival, photo, otp, completed, exits, restarts) are annotated with the nearest tracking battery reading within ± 60 min (`batteryAtTime`).
- **Photo / selfie uploads:** multer buffers to `uploads/task-photos` , the buffer is pushed to DigitalOcean Spaces, then the local temp file is deleted. Selfies are foldered by `YYYY-MM` . Requires `DIGITAL_OCEAN_*` env vars.
- **OTP:** 4-digit, emailed to the customer via `sendTaskOtpEmail` (SendPulse — needs `SENDPULSE_*` env). Stored in `task_details.otpCode` ; valid conceptually 10 min (not hard-enforced server-side).
- **End** (`endTask`): only valid from `arrived` / `holdOnArrival` / `reopenedOnArrival` ; idempotent if already done; chooses `completed` vs `waiting_for_approval` based on `requireApprovalOnComplete` ; persists final travel metrics.
- **Live location** (`updateLocation`): does **not** persist — it only emits over Socket.io to rooms `task:<id>` , `admin:tracking` , and `admin:staff:<staffId>` . Persistence happens via the separate `POST /tracking/store` .

3. Mobile app (Flutter — hrms)

3.1 Files

Layer	File	Role
Model	<code>lib/models/task.dart</code>	Task, TaskStatus, locations, exit/restart/destination records, TaskCompletionReport, timeline/route
Service	<code>lib/services/task_service.dart</code>	All HTTP calls + offline tracking queue
Repository	<code>lib/repository/task_repository.dart</code>	Thin abstraction over service (+ customer)
Bloc	<code>lib/bloc/task/</code>	<code>task_bloc</code> / <code>task_event</code> / <code>task_state</code>
Widget	<code>lib/widgets/task_card.dart</code>	Task list card
Screens	<code>lib/screens/geo/</code>	See below

Screens (`lib/screens/geo/`): `my_tasks_screen` (list), `add_task_screen` (create), `task_detail_screen` , `task_next_steps_screen` , `task_selfie_screen` , `end_task_screen` , `task_completed_screen` , `completed_task_detail_screen` , `task_history_screen` .

3.2 Status enum mapping

`TaskStatus` (app) ↔ backend strings via `Task.statusToApiString` / `Task.statusFromJson` . Snake_case on the wire (`in_progress` , `waiting_for_approval`); unknown values fall back to `TaskStatus.onlineReady` . `statusFromJson` mirrors the backend `STATUS_GROUP_SPELLINGS` so the card badge matches the server-side filter group.

3.3 Offline GPS queue (in TaskService)

`storeTracking()` posts every live point to `POST /tracking/store`. On a network failure the point is queued in `SharedPreferences` (`task_tracking_offline_queue_v1`) and a 1-minute periodic timer drains it in batches (20, dropping to 10 after a network failure). An outlier filter (`TrackingOutlierFilterService`) can skip jumpy/implausible points before sending. This is the durability layer behind the live-tracking map.

3.4 Typical client flow

1. Create task → `createTask` (`POST /tasks`).
2. Start ride → `updateTask(status: 'in_progress', startLat/Lng)` then begin `storeTracking` every ~15 s.
3. (Optional) exit/hold → `exitRide`; resume → `restartTask`.
4. Arrived → `arrivedRide` (`POST /tracking/arrived`, sets `arrived`).
5. Prove visit → `uploadPhotoProof` / `uploadTaskSelfie` / `submitFormResponse` / `sendOtp` + `verifyOtp`, each flipping a progress step.
6. Complete → `endTask` (`POST /tasks/:id/end`).
7. Review → `getTaskCompletionReport` renders the timeline + route polyline.

4. External dependencies & env

Dependency	Used for	Env vars
DigitalOcean Spaces	Photo proof & selfie storage	<code>DIGITAL_OCEAN_ACCESS_KEY</code> , <code>DIGITAL_OCEAN_SECRET_KEY</code> , bucket
SendPulse (email)	Customer OTP delivery	<code>SENDPULSE_CLIENT_ID</code> , <code>SENDPULSE_CLIENT_SECRET</code> , <code>SENDPULSE_FROM_EMAIL</code>
Socket.io	Live location broadcast to staff/admin	—
MongoDB	<code>tasks</code> , <code>task_details</code> , <code>task-settings</code> , <code>trackings</code>	<code>MONGO_URI</code> / <code>MONGODB_URI</code>

5. Gotchas

- **Don't store extended fields on `tasks`**. Any `PATCH /tasks/:id` unsets them; write detail through `task_details` / `upsertTaskDetails`.
- `task_details.taskId` is the **ObjectId**, not `TASK-...`. Mixing them silently breaks lookups.
- **Travel metrics are derived, not authoritative**. They are recomputed from `trackings` on read/end; a client-sent value is only a fallback.
- **OTP-required and approval flags come from `task-settings`**, not the task — set them per company.
- **Status spellings are dual-maintained** in `STATUS_GROUP_SPELLINGS` (backend) and `Task.statusFromJson` (app). Update both together or filters/badges diverge.